

Bản trình chiếu: Quy hoạch cân bằng

Zheng Dun

2008

Nguồn gốc: Quy hoạch cân bằng: bản trình chiếu

IOI China candidate team papers / 2008 / Day 1 / Zheng Dun / balanced planning.ppt

1 Tư tưởng cân bằng

Slide mở đầu từ một nhận xét thực dụng: giải bài bằng máy tính không chỉ là chọn thuật toán có độ phức tạp đẹp nhất. Trong phòng thi, ta còn phải cân bằng thời gian cài đặt, xác suất đúng, bộ nhớ, hằng số chạy và mức độ rủi ro.

“Quy hoạch cân bằng” trong bài này là kỹ năng lựa chọn điểm đứng giữa các yếu tố đó. Một lời giải rất mạnh nhưng khó viết có thể kém hiệu quả hơn một lời giải đơn giản, nếu lời giải đơn giản đủ qua dữ liệu hoặc đủ lấy điểm từng phần. Ngược lại, một tối ưu cục bộ có thể đáng làm nếu nó nâng độ chắc chắn mà không làm chương trình phình quá lớn.

2 Hiện thực không điển hình của thuật toán kinh điển

Ví dụ thứ nhất là *URAL 1099*. Bản chất bài là ghép cặp cực đại trên đồ thị tổng quát; thuật toán chuẩn là blossom. Nhưng blossom dài và dễ sai, trong khi giới hạn dữ liệu chỉ 222 đỉnh. Slide vì thế chọn một cách cân bằng: dùng tìm cây tăng luân phiên kiểu đơn giản, thêm thứ tự ngẫu nhiên và chạy nhiều lần.

Thuật toán không có bảo đảm tuyệt đối như blossom, nhưng các dữ liệu làm nó sai thường phụ thuộc mạnh vào thứ tự mở rộng. Khi xáo trộn thứ tự đỉnh và thứ tự mở rộng, liên hệ ấy bị phá vỡ. Để tránh chạy lại toàn cục quá nhiều, slide thêm ngưỡng thất bại: nếu tìm đường tăng cho một đỉnh chưa ghép thất bại, đổi thứ tự và thử lại vài lần rồi mới bỏ qua đỉnh đó.

Bảng thí nghiệm trong bài Word cho thấy các tham số như $5 - 5$, $5 - 10$, $20 - 1$, $50 - 1$ tạo ra cân bằng khác nhau giữa xác suất AC và thời gian. Ý nghĩa của ví dụ không phải khuyến khích thay blossom bằng may rủi trong mọi bài, mà là chỉ ra khi nào một thuật toán kinh điển quá nặng có thể được thay bằng một biến thể thực dụng hơn.

3 Cân bằng hằng số trong hiện thực

Ví dụ *Jackpot* (PKU 2103) cần tính xác suất bằng phân số lớn. Công thức chính dùng bao hàm - loại trừ trên tối đa 16 số, nhưng nút thắt nằm ở phép toán số lớn.

Nếu lưu số lớn theo cơ số 10^m , việc in kết quả thập phân rất tiện, nhưng phép nhân/cộng phải dùng nhiều phép chia và lấy dư. Nếu lưu theo cơ số 2^l , ta thay chúng bằng dịch bit và phép and; thao tác chính

nhanh hơn, còn bước in thập phân khó hơn một chút. Đây là một cân bằng có lợi vì số lần tính toán lớn hơn nhiều so với số lần in.

Thông điệp của slide: đôi khi thuật toán đã đúng, nhưng hiện thực chưa cân bằng. Đổi cơ số lưu trữ, chọn kiểu số nguyên, hoặc chuyển chi phí sang phần ít chạy hơn có thể quyết định việc qua thời gian.

4 Thuật toán không hoàn hảo

Ví dụ *NOI 2007 Catch* thuộc nhóm có thể lấy điểm bằng lời giải không hoàn hảo. Khi chưa tìm được thuật toán tối ưu, ta vẫn có thể xây dựng phương án dựa trên quy hoạch động, tham lam hoặc giả định cấu trúc để đạt một phần đáng kể số điểm. Điểm cần cân bằng là: thời gian bỏ ra cho lời giải từng phần có đáng so với lợi ích trên bảng điểm hay không.

Slide đặt nhóm này cạnh các ví dụ ngẫu nhiên như Treap. Treap không đảm bảo cân bằng tuyệt đối trong mọi lần chạy, nhưng nhờ ưu tiên ngẫu nhiên, chiều cao kỳ vọng nhỏ và cài đặt ngắn. Ta chấp nhận xác suất rủi ro rất thấp để đổi lấy độ đơn giản và hằng số tốt.

5 Cấu trúc hóa bài toán phức tạp

Phần sau của bài viết dùng bài duy trì cây PKU 3237 làm ví dụ. Trên cây có các thao tác đổi trọng số cạnh, đổi dấu trên đường đi và hỏi giá trị lớn nhất trên đường. Cấu trúc dữ liệu đầy đủ như dynamic tree có thể giải bài toán, nhưng khó cài trong bối cảnh thi.

Cách cân bằng là phân rã cây thành các dây, dùng LCA để tách đường A đến B thành vài đoạn trên dây, rồi đặt cây đoạn lên từng dây. Khi xử lý NEGATE hay QUERY, ta nhảy qua các dây và thực hiện thao tác đoạn quen thuộc. Cách này không tổng quát bằng Link-Cut Tree, nhưng dễ viết hơn và đủ nhanh cho dữ liệu.

6 Kết luận của slide

Quy hoạch cân bằng là một thói quen thiết kế:

- dùng thuật toán chuẩn khi cần bảo đảm và có đủ thời gian cài;
- dùng biến thể đơn giản hơn khi rủi ro được kiểm soát;
- đổi bộ nhớ lấy thời gian, hoặc đổi hằng số ở phần chạy nhiều lấy chi phí ở phần chạy ít;
- chọn cấu trúc dữ liệu vừa đủ mạnh thay vì luôn chọn cấu trúc tổng quát nhất.

Trong thi tin học, nhiều lời giải tốt đến từ việc tìm đúng điểm cân bằng giữa lý thuyết, hiện thực và dữ liệu.